

Parameters vs. Arguments

A **Parameter** is a variable defined in the method signature. An **Argument** is the actual value passed during the call.

Definition: `static void greet(String name)` → *name* is the parameter.

Execution: `greet("Alice")` → "Alice" is the argument.

Multi-Parameter Signature

Methods can accept multiple inputs of different types. The order and data type must match exactly when calling.

```
static void login(String user, int pin, boolean isAdmin) {  
    System.out.println("User: " + user + " | Admin: " + isAdmin);  
}  
  
// Calling with specific arguments  
login("JohnDoe", 1234, true);
```

Data Flow Logic

Parameters are used to drive internal logic, such as conditional checks or complex bank transactions.

```
static void withdraw(double balance, double amount) {  
    if (amount <= balance) {  
        System.out.println("New Balance: " + (balance - amount));  
    } else {  
        System.out.println("Denied: Insufficient Funds");  
    }  
}
```

Array as a Parameter

You can pass entire collections of data into a method using array parameters.

```
static void processAges(int[] nums) {  
    for (int n : nums) {  
        System.out.println("Processing: " + n);  
    }  
}  
  
int[] data = {21, 30, 18};  
processAges(data);
```

Parameter Rules

- **Exact Match:** The number of arguments must equal the number of parameters.
- **Type Safety:** Java will throw an error if you pass a String into an int parameter.
- **Pass-by-Value:** Java passes a *copy* of the value to the method (for primitives).